

Class 04 — Student Notes

Bong Study Hub — Foundation Batch 2026

How Computers Represent Information

From Information → Bits → Binary

These notes summarize what we covered in class. Use them to revise, not to learn the topic for the first time — the real learning happened in the room, through the questions and the invent-your-own-code activity. No electronics, no circuits, and no programming appear anywhere in this class — everything here is conceptual.

The Big Question

Computers work with all kinds of information — numbers, text, images, sound, video. But a computer can't just "look at" a photograph or understand a sentence the way a person does. Something has to happen first.

"If computers work with information like text, images, and sound, how do they actually represent that information?"

We won't answer this in one line. We'll build toward it, the same way we built toward "what is data?" in Class 03.

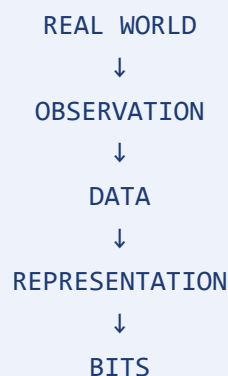
Quick connection to Class 03

Last class, we agreed on this:

DATA = RECORDED INFORMATION ABOUT SOMETHING

Class 03 asked *"what is data?"* Class 04 asks the next question:

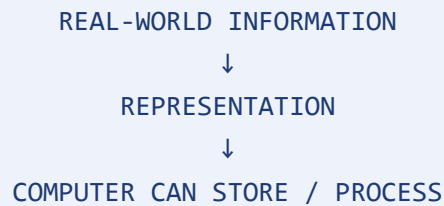
"how does a computer actually represent that data?"



By the end of today, every arrow in that chain will make sense.

1. Information Needs Representation

A computer cannot simply "look at" a photograph, or understand a number or a sentence, the way a human does. Before a computer can store or process any information, that information has to be turned into some form the computer can actually hold onto.



This doesn't mean humans and computers "understand" information in the same way — they don't. It just means that whatever a computer works with, it first needs some stored form to work with at all.

Think about it: How can one machine — the same hardware — end up representing numbers, letters, photographs, sound, *and* video?

Keep that question in mind. We're about to build the answer piece by piece.

2. A Simple Idea: Two States

Before talking about computers, think about a few everyday systems:

- A light switch: **off / on**
- A yes/no question: **no / yes**
- A door: **closed / open**

Each of these works perfectly well using just two clearly distinguishable states. You never get confused about whether a light switch is "sort of on."

DIGITAL = information represented using distinct, clearly separate states — rather than a smooth, continuous range.

A dimmer switch, by contrast, isn't like this — it can sit anywhere across a whole range. Computers are built the *first* way: from many small pieces that each reliably sit in one of a small number of clear states.

3. What Is a Bit?

BIT = **B**inary **D**igi**T**

A bit represents **one of two possible states**. We write those two states as:

0
1

ONE BIT

0 | 1

two possible states

Important: a bit is not a special kind of number. It's just a label for "one of two states." We could just as easily have called the two states off/on, or no/yes — 0 and 1 are simply the conventional symbols everyone agreed to use.

4. Why Two States?

Why did computers settle on exactly *two* states, instead of ten states like our everyday decimal numbers use?

A system with two clearly, reliably distinguishable states is much simpler to build and combine — over and over, billions of times — than a system that has to reliably tell apart ten subtly different states every single time.

Remember: Binary is a *design choice* for digital systems — it does not mean the real world itself is naturally binary.

A better way to say it:

"Digital computers represent information using combinations of binary states."

...rather than:

~~"Computers only understand 0 and 1."~~

5. One Bit → Two Possibilities

1 bit

↓

2 possible states (0 or 1)

One bit is genuinely limited — it can only ever be one of two things. That's exactly enough for a single yes/no answer, but not much more.

Think about it: What's a real, everyday message that only needs two possible options?

So — what happens if we use more than one bit at a time?

6. More Bits → More Possibilities

Take **two** bits instead of one. Each one is still just 0 or 1, but now we can list every possible combination of the two, together:

```
00
01
10
11
```

That's **four** possible combinations — not two more, but *double* the one-bit case.

```
1 bit → 2 combinations
2 bits → 4 combinations
3 bits → 8 combinations
```

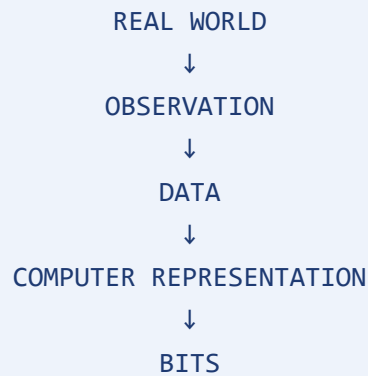
Each extra bit roughly **doubles** how many combinations you can make. You don't need a formula to feel this — just notice that everything you had before gets paired with both a 0 *and* a 1 for the new bit, so it doubles.

Remember: more bits means more possible *combinations*. Whether that actually makes a representation better depends on how those combinations are used — not just how many there are.

7. Binary Is a Representation System

A pattern like 01 doesn't mean anything **by itself**. It *represents* something — the same way a number or a word represents something.

Remember Class 03's canteen example? "80% full" was our recorded *representation* of a crowded canteen — not the canteen itself. Binary works exactly the same way: a bit pattern is a representation of information, not the information itself, and definitely not the real-world thing the information is about.



8. Many Kinds of Information, Same Building Block

Now the surprising part. All of these very different kinds of information can, in principle, be represented using combinations of the very same building block — bits:



A number becoming a bit pattern feels believable. A letter, or a whole word, getting its own bit pattern feels believable too. A photograph or a song feels much harder to picture — and it's fair for that to still feel a little mysterious. The short version: something like a photograph gets broken into many, many tiny pieces, and each piece gets its own bit pattern. *Exactly how* that works is a story for a later class — today's point is simply that totally different kinds of information can share the exact same underlying building block.

9. Same Bits, Different Interpretation

Here's a question worth sitting with: **does a sequence of bits carry its own meaning, or does something else decide what it means?**

Think about the marks "S O S." Those three letters mean something very specific — a distress signal — because we've all agreed on that meaning. In a different context, the same three letters could just be someone's initials. The marks didn't change. What changed was the

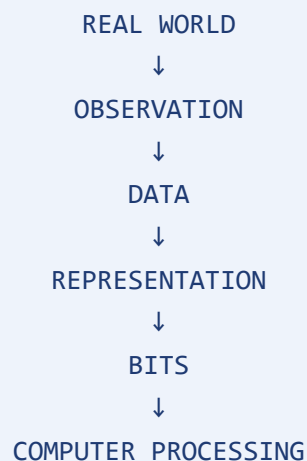
system used to interpret them.

A sequence of bits has meaning because we agree on how to interpret it. The pattern 01000001 means nothing on its own — it only means something once some system says "when you see this pattern, treat it as ____."

This is exactly what you'll do in the activity below.

10. Connecting It All

Let's put the whole day on one chain — starting all the way back at Class 03:



The top three lines are exactly what Class 03 taught. Today extended the chain: data doesn't just sit there — it has to become a representation, and that representation is built out of bits, before a computer can process it at all.

Try It Yourself — Invent a Tiny Binary Code

You have **2 bits** to work with — four possible combinations:

00
01
10
11

Invent a meaning for each one. For example, you could use them for four colors:

Bits	Meaning
00	Red
01	Green
10	Blue

11	Yellow
----	--------

Now ask yourself: **did the bits themselves contain that meaning, or did you just decide it?**
That's the whole idea from Section 9, in your own hands.

Quick Recap

- Computers can't "just understand" real-world information — it has to be turned into a **representation** first.
- Digital systems are built from clearly distinguishable **states**, like a light switch's off/on.
- A **bit** is one of two possible states, written 0 or 1. It is not a special kind of number.
- **One bit** gives two possibilities. **More bits** give more possible combinations — roughly doubling with each extra bit.
- **Binary is a representation system** — a bit pattern represents information; it isn't the information itself, and definitely isn't the real-world thing.
- Very different kinds of information — numbers, text, images, sound, video — can all be represented using the same underlying building block: bits.
- A bit pattern has no meaning on its own — **meaning comes from an agreed interpretation**.
- Class 04's chain connects directly onto Class 03's: [REAL WORLD](#) → [OBSERVATION](#) → [DATA](#) → [REPRESENTATION](#) → [BITS](#) → [COMPUTER PROCESSING](#).

Key Terms

Term	Meaning
Information	Anything meaningful we might want to record or work with — a number, a word, a picture, a sound.
Representation	A stored form that stands in for real information so a computer can hold onto it.
Digital	Built from a limited number of clearly distinguishable states, rather than a smooth continuous range.
Bit	One binary digit — a single unit that can be in one of two possible states (0 or 1).
Binary	A representation system built entirely from two-state units (bits).
Interpretation	The agreed-upon rule for what a given pattern of bits is supposed to mean.

Think About It

Try answering these in your own words — you don't need exact wording, just the right idea.

1. Why can't a computer just "understand" a photograph the way a person does?
2. What is a bit, in your own words?
3. How many combinations do 2 bits give? What about 3?
4. Is a binary pattern the same thing as the information it represents?
5. Name two very different kinds of information that could both be represented using bits.
6. Where does the *meaning* of a bit pattern actually come from?

Final Takeaway

"Computers do not need a different kind of machine for every kind of information. They can build many kinds of digital information from the same basic building block: bits."